

WEEK – 1, 2

Fundamentals of Object Oriented Programming: Object oriented paradigm, Object and Classes, Data Abstraction and Encapsulation, Inheritance, Polymorphism, Dynamic Binding.

1. Sample Case study on class & objects CO1

Bank Account Management System:

Problem Statement: Design a system to manage bank accounts.

Classes: Account, Savings-Account, Current-Account.

Attributes: Account number, balance, account type, interest rate (for savings account), overdraft limit (for current account), etc.

Methods: Deposit, Withdraw, Calculate Interest, Transfer, etc.

Object Usage: Create objects for each customer's account and perform transactions like deposits, withdrawals, and transfers.

Steps For Execution:

1. Create a folder named BankAccManagement.
2. Inside it, create a folder named record.
3. Save the Java program as BankAccManagement.java inside the record folder.
4. Open the BankAccManagement folder in IntelliJ IDEA/VS Code/Eclipse
5. If there are no compilation errors, execute the program.
6. The program creates a SavingsAccount with account number, balance, and interest rate.
7. The program displays "Savings Account".
8. It deposits 1800 into the savings account.
9. It withdraws 100 from the savings account.
10. The program calculates the interest and adds it to the balance.
11. The savings account details are displayed.
12. The program creates a CurrentAccount with account number, balance, and overdraft limit
13. It deposits 1000 into the current account.
14. The program attempts to withdraw 3000.
15. The program checks whether the withdrawal amount is within the balance and overdraft limit.

16. Since 3000 exceeds the allowed overdraft limit, the withdrawal is rejected.
17. The current account details are displayed.
18. The program execution is completed.

ALGORITHM: Bank Account Management

Step 1: Start.

Step 2: Create a SavingsAccount object with:

Account Number = 2347566

Balance = 99000

Interest Rate = 9.0%

Step 3: Display "Savings Account".

Step 4: Deposit 1800 into the savings account.

New Balance = $99000 + 1800 = 100800$

Step 5: Withdraw 100 from the savings account.

New Balance = $100800 - 100 = 100700$

Step 6: Calculate interest.

Interest = $100700 \times 9 / 100 = 9063$

Step 7: Add the interest to the balance.

New Balance = $100700 + 9063 = 109763$

Step 8: Display the savings account details.

Step 9: Create a CurrentAccount object with:

Account Number = 44564

Balance = 300

Overdraft Limit = 567

Step 10: Deposit 1000 into the current account.

New Balance = 300 + 1000 = 1300

Step 11: Try to withdraw 3000.

Step 12: Check whether:

Withdrawal Amount <= Balance + Overdraft Limit

3000 <= 1300 + 567

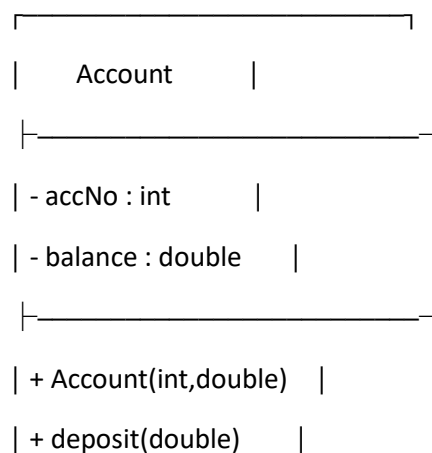
3000 <= 1867 → False

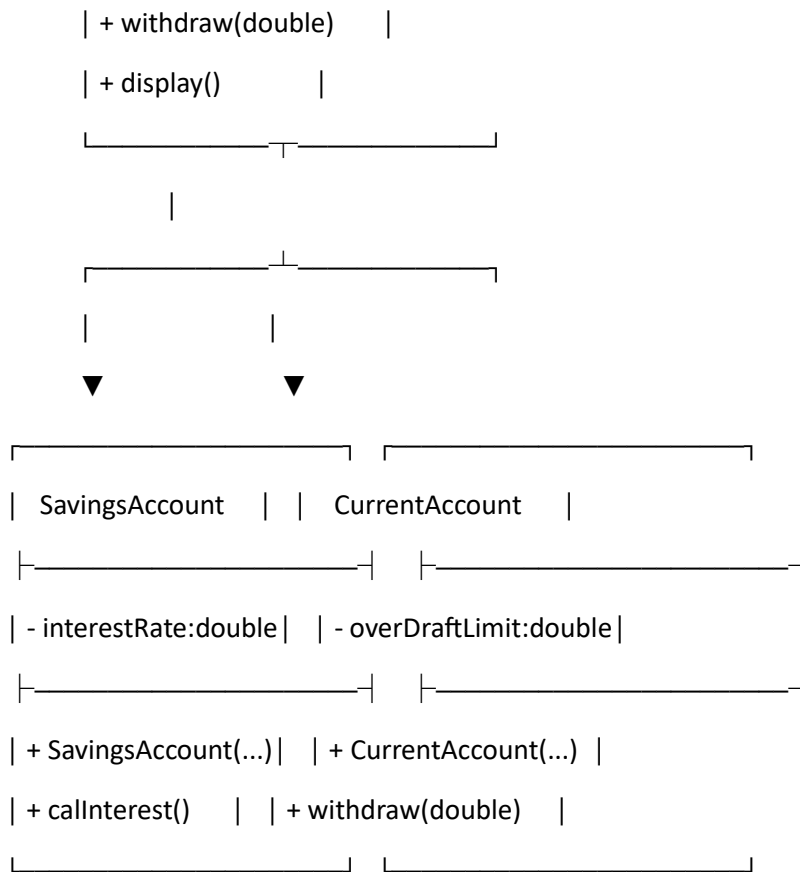
Step 13: Display "Overdraft limit exceeded".

Step 14: Display the current account details.

Step 15: Stop.

UML Diagram





Code

```

package javacore;
class Account {
    int accNo;
    double balance;
    Account (int accNo,double balance){
        this.accNo=accNo;
        this.balance=balance;
    }
    void deposit(double amount){
        balance=balance+amount;
        System.out.println("Deposited: "+amount);
    }
    void withdraw(double amount) {
        if (amount <= balance) {
            balance = balance-amount;
            System.out.println("Withdrawn: " + amount);
        } else {
            System.out.println("Insufficient-Amount can't be taken");
        }
    }
}
  
```

```

    }
    void display() {
        System.out.println("Account No:" + accNo);
        System.out.println("Balance:" + balance);
    }
}

class SavingsAccount extends Account{
    double interestRate;
    SavingsAccount(int accNo,double balance,double interestRate){
        super(accNo, balance);
        this.interestRate=interestRate;
    }
    void callInterest(){
        double interest=balance*interestRate/100;
        balance=balance+interest;
        System.out.println("Interest:"+interest);
    }
}

class CurrentAccount extends Account{
    double overDraftLimit;
    CurrentAccount(int accNo,double balance,double overDraftLimit) {
        super(accNo,balance);
        this.overDraftLimit=overDraftLimit;
    }
    void withdraw(double amount){
        if(amount<=balance+overDraftLimit){
            balance=balance-amount;
            System.out.println(("Withdrawn: "+amount));
        }
        else{
            System.out.println("Overdraft limit exceded");
        }
    }
}

public class BankAccManagement {
    public static void main(String[] args){
        SavingsAccount sa=new SavingsAccount(2347566,99000,9.0);
        System.out.println("Savings Account");
        sa.deposit(1800);
        sa.withdraw(100);
    }
}

```

```
        sa.callInterest();
        sa.display();
        System.out.println();
        CurrentAccount c= new CurrentAccount(44564,300,567);
        c.deposit(1000);
        c.withdraw(3000);
        c.display();

    }
}
```

Output:

Savings Account

Deposited: 1800.0

Withdrawn: 100.0

Interest:9063.0

Account No:2347566

Balance:109763.0

Deposited: 1000.0

Overdraft limit exceeded

Account No:44564

Balance:1300.0